



Ariel Soto

Escuela de Administración y Negocios
Universidad de Concepción · Campus Chillán

SEMESTRE 1 · 2026



Fundamentos de Programación para Análisis Económico

MÓDULO II · SEMANA 6 · SESIÓN 2

Datos Faltantes y Outliers

Los datos reales vienen rotos

Hasta hoy trabajamos con archivos que había preparado para ustedes. Esta sesión usa uno que **nadie preparó**: la Encuesta Nacional de Empleo del INE, tal como se descarga.

Ignorar los faltantes no es opción; tratarlos mal **sesga** el análisis. Y algunos ni siquiera parecen faltantes.

Al terminar esta sesión podrás:

- ▶ **Abrir** un archivo público tal como lo entrega la institución
- ▶ Distinguir `NA`, `NaN` e `Inf`, y separar el faltante **real** del **estructural**
- ▶ Detectar y **medir** los faltantes antes de actuar
- ▶ Decidir entre **eliminar e imputar** con criterio
- ▶ Identificar **outliers** y **centinelas** — documentando todo

Bloque A — Abrir un archivo real



Hoy no usamos un archivo de juguete

La **Encuesta Nacional de Empleo** del INE, trimestre móvil mayo–julio 2026. Es el dato con que Chile calcula su tasa de desocupación:

```
# Se descarga de ine.gob.cl y se guarda en data/raw/  
ene <- read.csv("../data/raw/ene-2026-06-mjj.csv")  
ncol(ene)
```

```
[1] 1
```

Una columna. No es que el archivo esté roto: `read.csv()` supone que el separador es la coma, y el INE usa **punto y coma**. Todo el archivo entró como una sola cadena de texto por fila.

El separador y la coma decimal

```
ene <- read.csv2("../data/raw/ene-2026-06-mjj.csv")  
dim(ene)
```

```
[1] 97946 222
```

`read.csv2()` es la variante europea: separador `;` y **decimal con coma**. Ambas cosas importan. Si abren un archivo con coma decimal usando `read.csv()`, los números llegan como texto y `mean()` falla sin explicar por qué.

Regla práctica: **abran el archivo en un editor de texto antes de leerlo en R**. Mirar las dos primeras líneas ahorra media hora de depuración.

222 columnas: aquí sirven los selectores

Nadie trabaja con 222 variables a la vez. Los selectores de la **Semana 5** dejan de ser una comodidad y pasan a ser indispensables:

```
names(select(ene, starts_with("cae")))
```

```
[1] "cae_general"  "cae_especifico"
```

```
# Quedarse con lo que la pregunta necesita
empleo <- select(ene, region, sexo, edad, activ,
                 habituales, efectivas, fact_cal)
dim(empleo)
```

```
[1] 97946      7
```

`activ` es la condición de actividad (1 ocupado · 2 desocupado · 3 inactivo), `habituales` y `efectivas` son horas semanales, y `fact_cal` es el **factor de expansión**: cuántas personas de Chile representa esa fila.

`activ` es la **columna 221 de 222**: está al final, en el bloque de indicadores que el INE **construye** (`pet`, `ft`, `activ`, `cae_general`, `fact_cal`). Las columnas `b1`, `b2` ... son **preguntas** del cuestionario, sección B, que solo responden los ocupados — `b1` es el grupo de ocupación, no si trabaja o no.

Dos calendarios en un mismo archivo

La ENE es un **trimestre móvil**, y por eso trae dos pares de columnas de tiempo que no son redundantes:

```
table(ene$mes_central)

      6
97946

table(ene$mes_encuesta)

      5      6      7
32337 32752 32857
```

Par de columnas	Qué responde	En esta base
ano_trimestre / mes_central	¿a qué trimestre pertenece la base?	siempre 2026 / 6 = mayo–julio
ano_encuesta / mes_encuesta	¿en qué mes entrevistaron a esta persona?	5, 6 o 7, un tercio cada uno

El nombre del archivo lo codifica: ene-2026-06-mjj = año 2026, trimestre **06**, iniciales **mayo-junio-julio**. La “tasa de junio” que sale en la prensa es en realidad la de mayo–julio.

Por qué el INE junta tres meses

Si cada mes se entrevista a ~32.000 personas, ¿por qué no publicar mes a mes? El caso de Aysén nos da la idea:

```
ft <- filter(ene, activ %in% 1:2)
ft |> filter(region == 11) |> group_by(mes_encuesta) |>
  summarise(n = n(), tasa = round(100 * sum(fact_cal[activ == 2]) / sum(fact_cal), 2))
```

```
# A tibble: 3 × 3
  mes_encuesta     n  tasa
    <int> <int> <dbl>
1         5   370  4.33
2         6   382  3.89
3         7   364  3.71
```

~370 personas por mes, y la tasa salta de 4,33 a 3,71 sin que haya cambiado nada real: es ruido de muestreo. El trimestre completo son 1.116 personas y **3,96 %**.

El trimestre móvil resuelve	Cómo
Precisión	tres meses triplican la muestra; las regiones chicas tienen cifra publicable
Oportunidad	un trimestre cerrado daría 4 datos al año; la ventana móvil da 12
Operación	el campo es continuo: cada mes se visita a un tercio de la muestra

El costo es el **solapamiento**: dos trimestres consecutivos comparten dos meses y no son observaciones independientes. Y por eso hay dos años: el trimestre diciembre–febrero cruza el cambio de año, y `ano_encuesta` registra el real de cada entrevista.

Bloque B — Tipos de faltantes



NA , NaN , Inf : no son lo mismo

Valor	Qué significa	De dónde sale
NA	dato ausente (desconocido)	no respuesta, celda vacía
NaN	"not a number" (indefinido)	<code>0/0</code> , <code>log(-1)</code>
Inf	infinito	<code>1/0</code> , dividir por cero

```
c(0/0, 1/0, -1/0) # NaN, Inf, -Inf
```

```
[1] NaN Inf -Inf
```

```
NA + 5 # NA → el faltante se "contagia"
```

```
[1] NA
```

Cualquier operación con `NA` da `NA` . Por eso `mean()` sin `na.rm` devuelve `NA` (Semanas 4–5). A veces el `NA` está **disfrazado** de número: `99` , `9999` , `-9` .

El faltante que NO es un problema

La ENE tiene 15.305 personas sin condición de actividad. Antes de imputar nada, hay que preguntarse **quiénes son**:

```
table(ene$activ, useNA = "ifany")
```

```
  1      2      3 <NA>
41850 4412 36379 15305
```

```
summary(ene$edad[is.na(ene$activ)]) # ¿qué edad tienen los "faltantes"?
```

```
  Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
 0.000   5.000   9.000   8.105  12.000  14.000
```

Ninguno pasa de 14 años. No falta el dato: la pregunta **no se les hizo**, porque no están en edad de trabajar. Eso es un faltante **estructural**.

Estructural contra real: la distinción que cambia todo

	Faltante estructural	Faltante real
Qué pasó	la pregunta no aplica	había respuesta y se perdió
Ejemplo ENE	<code>activ</code> en un niño de 8 años	ingreso que el hogar no declara
Qué se hace	excluir del denominador	eliminar o imputar, con criterio
Si se trata mal	se inventa población	se sesga el resultado

Imputar un faltante estructural es un error grave: equivale a afirmar que un niño de 8 años está desocupado. La tasa de desocupación **no se calcula sobre las 97.946 filas**, sino sobre quienes están en la fuerza de trabajo.

💬 La pregunta que hay que hacerse siempre ante un `NA` : *¿esto se perdió, o nunca existió?* La respuesta decide el tratamiento.

Bloque C — Detectar faltantes



`is.na()` : la herramienta base

```
sum(is.na(empleo$habituales))    # CUÁNTOS faltan (patrón sum de S3)
```

```
[1] 56096
```

```
mean(is.na(empleo$habituales))  # qué PROPORCIÓN falta
```

```
[1] 0.5727237
```

Es el patrón `sum()` / `mean()` de una condición lógica (Semanas 3–4), ahora aplicado a los faltantes. **El 57 % de las filas no tiene horas trabajadas** — y eso no es un problema del archivo: son quienes no están ocupados.

Mapa de faltantes de toda la tabla

```
colSums(is.na(empleo))      # NA por columna: ¿dónde está el problema?
```

region	sexo	edad	activ habituales	efectivas	fact_cal
0	0	0	15305	56096	56096
					0

```
sum(complete.cases(empleo))  # filas sin NINGÚN NA
```

```
[1] 41850
```

Antes de limpiar, medir. Los faltantes no están repartidos al azar: se concentran en las dos columnas de horas, y siempre en las mismas filas.

Bloque D — Eliminar vs imputar



Eliminar filas con faltantes

```
nrow(na.omit(empleo))           # quitar toda fila con CUALQUIER NA
```

```
[1] 41850
```

```
table(na.omit(empleo)$activ)    # ¿quién sobrevivió?
```

```
1  
41850
```

Miren bien. `na.omit()` dejó 41.850 filas, y **todas son ocupados**. Borró a los 4.412 desocupados, porque un desocupado no tiene horas trabajadas. La tasa de desocupación de esa tabla es **0 %**.

Es el riesgo del método en su forma más pura: eliminar filas con faltantes borró exactamente al grupo que queríamos estudiar. `na.omit()` **sobre un archivo que no conoces es peligroso.**

La diferencia entre `na.omit()` y `complete.cases()` ** es que el primero borra las filas con valores `NA`, mientras que el segundo solo entrega valores lógicos, sin borrar.

Imputar: rellenar con un criterio

```
imputado <- empleo |>
  mutate(habituales = if_else(is.na(habituales),
                              median(habituales, na.rm = TRUE),
                              habituales))
sum(is.na(imputado$habituales))    # ya no quedan NA
```

```
[1] 0
```

Y acabamos de hacer una barbaridad. Le asignamos 42 horas semanales a 56.096 personas que **no trabajan**. El código corrió sin quejarse.

Imputar **inventa** datos y reduce artificialmente la varianza. Sobre un faltante **real** puede justificarse (mediana mejor que media, por robusta); sobre uno **estructural** nunca. Métodos avanzados (regresión, MICE) escapan a este curso: aquí, estrategia simple + **documentar**.

No hay respuesta única: hay criterio

La decisión —eliminar vs imputar— depende de **cuántos** faltan, **por qué**, y **para qué** es el análisis.

Lo único que **no** se puede hacer es ignorarlo en silencio.

Regla del curso: toda decisión sobre faltantes se documenta.

Bloque E — Outliers



¿Qué es un outlier y por qué importa?

Un valor extremo que se aparta del resto. Unos pocos muy altos **distorsionan la media** (por eso $\text{media} \neq \text{mediana}$, Semana 4).

```
summary(empleo$habituales)    # ¿el máximo es sospechoso?
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.	NAs
1.00	35.00	42.00	40.33	44.00	999.00	56096

No todo outlier es un error: 70 horas semanales son duras pero **posibles**. La pregunta no es “¿lo borro?” sino “¿es un error de datos o un caso legítimo?”

Detectar outliers: la regla del IQR

```
q <- quantile(empleo$habituales, c(.25, .75), na.rm = TRUE)
iqr <- q[2] - q[1]
tope <- q[2] + 1.5 * iqr
c(Q1 = q[[1]], Q3 = q[[2]], tope = tope[[1]])
```

```
Q1    Q3 tope
35.0 44.0 57.5
```

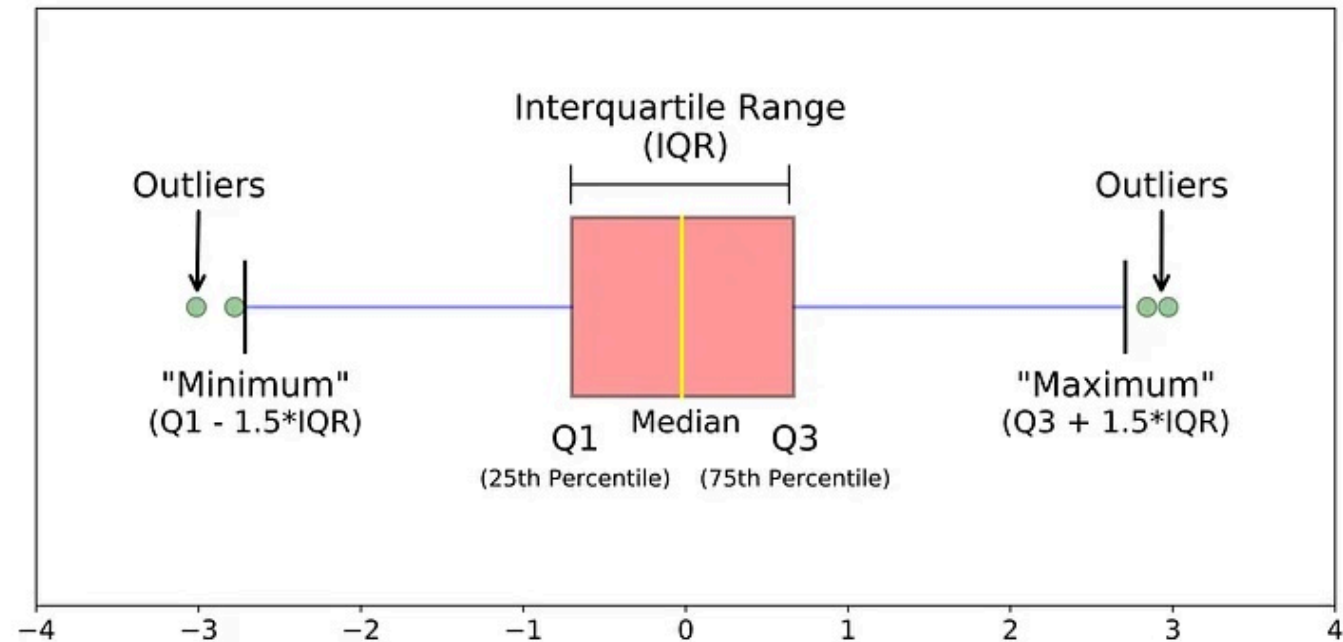
```
sum(empleo$habituales > tope, na.rm = TRUE) # candidatos a outlier
```

```
[1] 2123
```

Conecta `quantile()` (Semana 4) con un uso concreto: todo lo que supere **1,5×IQR** sobre el tercer cuartil es candidato. Aquí, más de 57,5 horas semanales.

Guarden ese 2.123. En dos slides van a descubrir que una parte de esos “outliers” **ni siquiera son datos**.

Usando el rango intercuantílico



El outlier que en realidad es un faltante

Las horas trabajadas de la ENE llegan hasta 999:

```
summary(ene$habituales)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.	NAs
1.00	35.00	42.00	40.33	44.00	999.00	56096

Nadie trabaja 999 horas a la semana: la semana tiene 168. El 999 es un **código centinela** — un faltante escrito como número. `is.na()` no lo detecta, y `mean()` lo suma como si fuera real.

```
horas <- ene$habituales[ene$activ == 1]
c(con_999 = mean(horas, na.rm = TRUE),
  sin_999 = mean(horas[horas != 999], na.rm = TRUE))
```

con_999	sin_999
40.32552	40.14223

Por qué hay que buscarlos a mano

El 999 son solo **8 casos**: mueven el promedio de 40,33 a 40,14 horas. Parece que no valía la pena.

Cuidado con quedarse ahí. `summary()` muestra el máximo, y el máximo era 999. Pero un centinela no tiene por qué ser el valor más grande: puede estar escondido más abajo en la cola, y ser **mucho más frecuente**.

```
# No mires solo el máximo: mira toda la cola
sort(table(ene$habituales[ene$habituales > 80]), decreasing = TRUE)
```

Eso lo hacen en el laboratorio, sobre este mismo archivo. Adelanto: **hay un segundo centinela**, y cuando lo limpien el promedio se mueve diez veces más que con el 999.

Los clásicos, por si aparecen: 99 · 999 · 9999 · 888 · -9 · -99 · NS/NR . La documentación de la encuesta dice qué significa cada uno; **leerla es parte del trabajo**.

Qué hacer con un outlier

Opciones: corregir (si es error tipográfico evidente) · winsorizar (topar al percentil 99) · excluir (justificado) · **dejarlo** (si es real).

El criterio sobre las horas de la ENE, ya sin centinelas:

- ▶ 999 y 888 → **códigos**, no jornadas → a NA con `na_if()`.
- ▶ 126 horas (18 al día, todos los días) → **implausible**, pero es lo que la persona declaró → se deja y **se advierte**.
- ▶ 60 horas → dura pero **real** en muchos oficios → se deja.

Decidir con criterio y dejar registro. **Nunca borrar en silencio**. Un dato implausible no es lo mismo que un dato imposible: el primero se discute, el segundo se corrige.

Bloque F — Documentar



El registro de limpieza

La limpieza es parte del análisis y debe ser **reproducible** (Semana 2). Un buen script documenta qué encontró y qué decidió:

```
# Bitácora de limpieza – ENE 2026 MJJ (INE), descargada el 10-09-2026
# – Lectura: read.csv2() -> separador ";" y coma decimal
# – Columnas: 222 originales -> 7 conservadas (select)
# – activ: 15.305 NA = menores de 15 -> ESTRUCTURAL, se excluyen (no se imputan)
# – habituales: centinelas 999 (8 casos) y 888 (103) -> a NA con na_if()
#           efecto en el promedio de ocupados: 40,33 -> 38,05 horas
# – habituales: máximo restante 126 h -> implausible, se conserva y se advierte
# – Ponderación: fact_cal, obligatoria para toda cifra reportada
# – Filas: 97.946 originales -> 97.946 finales (0 eliminadas)
```

Quien lea el script (o tú en seis meses) entiende **exactamente** qué se hizo y por qué. Eso es reproducibilidad.

Una fila no vale una persona

La ENE es una **muestra**. `fact_cal` dice cuántos chilenos representa cada fila: la primera vale por 1.079 personas, otra por 81.

```
ocupados    <- sum(ene$fact_cal[ene$activ == 1], na.rm = TRUE)
desocupados <- sum(ene$fact_cal[ene$activ == 2], na.rm = TRUE)
round(100 * desocupados / (ocupados + desocupados), 2) # tasa de desocupación
```

```
[1] 9.53
```

```
format(round(ocupados + desocupados), big.mark = ".") # fuerza de trabajo
```

```
[1] "10.295.061"
```

9,53 %. Ese es el número que sale en la prensa, y acaban de calcularlo desde el archivo crudo. La fuerza de trabajo son 10,3 millones de personas — no las 46.262 filas de la muestra.

El ponderador cambia el nivel, no siempre la tasa

Sin ponderar la tasa da **9,54 %**; ponderando, **9,53 %**. Casi lo mismo – y eso es una trampa.

La diferencia no está en la tasa, está en el **nivel**: sin `fact_cal` se reportarían **4.412 desocupados** en Chile, en vez de **981.044**.

Que un ponderador mueva poco una tasa **no autoriza a ignorarlo**. Cambia todo total, y mueve las comparaciones apenas los grupos están muestreados con intensidad distinta – las regiones pequeñas, por ejemplo.

Toda encuesta oficial de hogares trae ponderador: CASEN, ENE, EPF. Si el archivo tiene factor de expansión y no lo usan, su resultado **no es comparable con la cifra oficial**.

Cierre: lo que nos llevamos esta semana

Tema	En una línea
Dar forma	<code>pivot_longer</code> / <code>wider</code> , <code>separate</code> / <code>unite</code> → tidy data
Abrir lo real	<code>read.csv2()</code> : separador ; y coma decimal; selectores para 222 columnas
Tipos de faltante	<code>NA</code> (ausente), <code>NaN</code> , <code>Inf</code> — y estructural contra real
Detectar	<code>is.na()</code> , <code>colSums(is.na())</code> , <code>complete.cases()</code> , <code>summary()</code> para centinelas
Tratar	eliminar vs imputar — con criterio
Outliers	regla 1,5×IQR; el <code>999</code> que no es outlier sino código
Ponderar	<code>fact_cal</code> : una fila no vale una persona

Al laboratorio (2h): el ciclo completo sobre la **ENE del INE**, desde el archivo crudo hasta la tasa de desocupación por región. Descargar, abrir, recodificar, tratar centinelas y faltantes, ponderar y guardar. ChatGPT como **consultor**, no para decidir la estrategia.

A3 (formativa, sin nota): el laboratorio se entrega igual. Sin datos limpios no hay modelo: prepara la **T3** (Semana 8, calificada) y es el ensayo de lo que harán con sus propios datos en el **Proyecto Final**.



Sesión 2 — Fin

Al laboratorio práctico (2h)

Limpieza de un dataset real “sucio” · diagnosticar · reformatear ·
tratar faltantes y outliers · documentar

